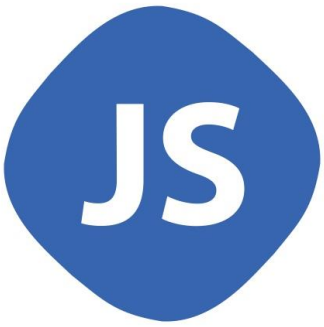


Введение в функции JS

- Введение
- Объявление и вызов функций
- Особенности функциональных выражений

Введение

В любом языке программирования существуют **пользовательские** функции. **Функция** - ключевая концепция в JavaScript. Функция — это специальным образом оформленный именованный фрагмент кода, к которому можно многократно обратиться из любого места внутри программы. После выполнения функции управление возвращается обратно в точку программы, где данная функция была вызвана.

В общем случае, функция — это "подпрограмма", которую можно вызывать из внешнего по отношению к функции кода.

> ***Важно***. **Функция не будет** выполняться сразу при загрузке страницы. Функция выполняется при ее **вызове** из тела программы.

Как и сама программа, функция состоит из **последовательности инструкций**, называемой **телом функции**.

> ***К сведению***. **Функция в JavaScript** это специальный тип **объекта** и, следовательно, ею можно манипулировать как объектом. Если быть точным, **функция** — это **объект function** (объекты - тема отдельного урока).

Функция может принимать **входные параметры.** Параметры, передаваемые функции при ее вызове называются **аргументами функции**.

Функция всегда **возвращает значение**. Чтобы вернуть значение, в функции должна быть инструкция **return**, которая указывает, что именно нужно вернуть. Функция **без return** вернёт значение по умолчанию, чаще всего значением по умолчанию будет **undefined**.

Таким образом:

Функция — это именованный фрагмент кода, к которому можно многократно обратиться из любого места внутри программы. Как правило, функция возвращает значение.

Процедура (функция-процедура) — это функция, которая возвращает пустое значение. При этом выполнение кода функции сказывается на выполнении программы или на состоянии DOM-документа.

Метод — это функция, которая является свойством объекта.

Объявлять функции в JavaScript можно с помощью синтаксиса:

- function **declaration**
- function **expression**

Объявление и вызов функций

Функция вида function declaration (функция инструкция)

Функция вида **function declaration** (или function definition или function statement) по синтаксису является **инструкцией**. Объявление функции состоит из ключевого слова **function** и следующих частей:

```
function name([param1, param2, ..., paramN])  {
     statements;
};
```

где:



- **function -** ключевое слово;
- **name -** имя функции;
- **paramN -** список параметров (принимаемых функцией) заключённых в круглые скобки и разделённых запятыми;
- **statements -** инструкции, которые будут выполнены после вызова функции, заключают в фигурные скобки **{ }**.

Пример **объявления** функции типа **declaration** с именем **square**:

```
// объявляем declaration функцию с именем square
function square (num) {
    return num * num;
}
```

Функция **square**() принимает параметр - **num**, и состоит из одного действия. Инструкция **return** указывает на значение, которое будет возвращено функцией.

Пример **вызова declaration** функции:

```
// вызываем функцию в теле программы
square (5); // 25
```

Ниже представлен пример объявления и вызова **declaration функции cube**().

example_1. Функция вида declaration / definition / statement

```
<script>
    // объявление declaration функции cube
    function cube(number) {
        let res = number * number * number;
        return res;
    }
    // вызов функции cube
```

```
    // функция возвращает значение переменной x
    let x = cube(3);
    console.log(x); // 27
</script>
```

Функция всегда возвращает значение, если инструкция **return** не используется, будет возвращено значение **undefined**. Пример использования функции без инструкции **return**.

example_2. Функция всегда что-то возвращает

```
<script>
    // объявление declaration функции cube
    function cube(number) {
        let res = number * number * number;
        // инструкцию return не используем
    }
    // вызов функции cube
    // функция возвращает значение в переменную x
    let x = cube(4);
    console.log(x); // undefined
</script>
```

В следующем демонстрационном примере инструкция **return** не используется и выполняется модификация дерева DOM документа.

example_3. Пример функции-процедуры (процедуры)

```
<script>
    // объявление declaration функции-процедуры fnProcedure
    function fnProcedure () {
```

```
        document.write("<h1>Добро пожаловать на
        pechora_PRO!</h1>");
        // функция не имеет инструкции return,
        // но модифицирует DOM документа
    }
    // вызов функции fnProcedure
    console.log(fnProcedure()); // undefined
</script>
```

Функция вида function expression (функциональное выражение)

Другой вид функции - **function expression** (функциональное выражение). В данном случае ключевое слово **function** может использоваться для определения функции внутри выражения. Такая функция может быть **анонимной** (не иметь имени).

```
var myFunction = function [name]([param1[, param2[, ..., paramN]]]) {
    statements;
};
```

где:

- **function** - ключевое слово;
- **name** - имя функции, является локальным для её тела, может быть опущено (в таком случае функция является анонимной). Пример использования **имени** рассмотрим дальше;
- **paramN** - имя аргумента, передаваемого в функцию;
- **statements** - инструкции (программный код), составляющие тело функции.

Пример **объявления** функции типа **expression** (анонимная функция):

```
// объявляем expression функцию
```

```
let square = function(num) {
    return num * num;
};
```

Пример **вызова expression** функции:

```
// вызываем функцию
var x = square (5);
console.log(x); // 25
```

Например, **expression функция cube**() может быть объявлена и вызвана следующим образом:

example_4. Функция вида expression (функциональное выражение)

```
<script>
    // объявление функции
    var cube = function (number) {
        result = number * number * number;
        return result;
    };
    // инициируем переменную value
    var value = 5;
    // передаем переменную value в качестве аргумента анонимной функции
    var x = cube(value);
    console.log(x);  // 125
</script>
```

Функции вида **expression** удобны, когда функция **передаётся аргументом другой функции**. Такой способ использования функции является весьма



распространенным среди разработчиков и будет часто применяться в следующих уроках.

example_5. Функция-процедура expression

```
<script>
     // объявление функции
     var myHello = function () {
          document.getElementById("console").firstChild.innerText
          = "Будь таким, каким хочешь казаться (Сократ)";
     };
     var el =  myHello();
     console.log(el);  // undefined
</script>
```

Функция вида arrow function expression (функция стрелка)

Существует ещё один **вариант синтаксиса** для создания **expression** функций, он называется функции-стрелки или **стрелочные функции** (arrow functions).

Пример **объявления** функции типа **arrow** (стрелочная функция):

```
// объявляем arrow функцию
let square = (num) => num * num;
```

Пример **вызова arrow** функции:

```
// вызываем функцию
 x = square(5);
console.log(x); // 25
```

Эта форма объявления создаёт стрелочную функцию, которая принимает параметр, затем вычисляет выражение в правой части и возвращает результат.



Стрелочная функция представляет собой более **короткий синтаксис функции expression**. Функцию **cube**() можно определить с использованием синтаксиса стрелки.

example_6. Однострочная стрелочная функция

```
<script>
    // объявление однострочной стрелочной функции cube()
    var cube = (num) => num * num * num;
    // вызов функции cube()
    var x = cube(3);
    alert(x); // 27
</script>
```

> ***Обратите внимание***. В однострочных стрелочных функциях инструкция **return** не используется.

Если у нас **только один аргумент**, то круглые скобки вокруг параметров **можно опустить**, сделав запись ещё короче:

```
// для одного параметра скобки можно опустить
var square = num => num * num;
```

Если **аргументов нет**, круглые скобки **будут пустыми**, но они должны присутствовать:

```
// если аргументов функции нет - пустые скобки должны присутствовать
var present = () => prompt("Представьтесь, пожалуйста");
```

Если вы хотите использовать стрелочную функцию с несколькими выражениями и инструкциями, необходимо использовать фигурные скобки. При этом важное

отличие – в том, что в таких скобках для возврата значения **необходимо использовать return** (как в обычных функциях).

example_7. Многострочная стрелочная функция

```
<script>
	// объявление многострочной стрелочной функции cube()
	// единственный аргумент num используем без скобок
	var cube = num => {
		let result = num * num * num;
		return result;
	};
	// вызов функции cube()
	var x = cube(3);
	alert(x); // 27
</script>
```

Так-же как предыдущие функции, стрелочная может быть процедурой.

example_8. Стрелочная функция-процедура

```
<script>
	// объявление многострочной стрелочной функции
	// аргументов нет – скобки нужны
	var present = () => {
		let login = prompt("Введите логин");
		let pwd = prompt("Введите пароль");
		if (login == "admin" && pwd == "12345"){
			document.write ("<h1>Добро пожаловать на
			pechora_PRO!</h1>");
```

```
        } else {
            document.write ("<h1>А вот вам, товарищ, вход
            запрещен!</h1>");
        }
    }
    // вызов многострочной стрелочной функции
    present();
</script>
```

Стрелочные функции очень удобны для выполнения простых однострочных действий.

Функция — это всегда значение

Независимо от того, как создаётся функция – она является **значением**. В обоих приведённых ниже примерах функция хранится в переменной **sayHi**.

```
// создаем переменную sayHi через function declaration
function sayHi() {
    alert( "Привет" );
}
// создаем переменную sayHi через function expression
let sayHi = function() {
    alert( "Привет" );
};
```

Значение переменной всегда можно вывести.

Обратите внимание, в коде демонстрационного примера **example_9** мы сначала выполняем функцию, затем **выводим ее код** сначала в браузер, затем в консоль. Код функции и есть значение переменной **discr**.

example_9. Функция — это значение

```
<script>
    // объявление стрелочной функции discr()
    var discr = () => {
        let b = 6;
        let a = 2;
        let c = 1;
        let d = b**2 - 4 * a * c;
        return d;
    };
    // вызов функции discr()
    console.log(discr());
    // вывод кода функции discr() в браузер
    document.write(discr);
    // вывод кода функции discr() в консоль
    console.log(discr);
</script>
```

Упоминание имени функции **без круглых скобок** приводит к выводу ее кода – значения переменной с именем функции. Приведённый выше код показывает строковое представление функции, которое является её исходным кодом.

Важно. В JavaScript функция – это значение, поэтому мы можем обращаться с ней как со значением.

Функция – это особое значение, в том смысле, что мы можем вызвать её, а можем просмотреть её исходным код. Но всё же это значение, поэтому мы можем работать с ней так же, как и с другими видами значений.



В следующем примере мы копируем исходный код функции в другую переменную.

example_10. Работа с функциями как с переменными

```
<script>
    // объявление функции discr()
    function discr() {
        let b = 6;
        let a = 2;
        let c = 1;
        let d = b**2 - 4 * a * c;
        return d;
    };
    // вызов функции discr()
    console.log(discr());
    // присвоение новой переменной исходного кода функции
    var new_var = discr;
    // вызов функции через новую переменную
    console.log(new_var());
    // вывод исходного кода функции
    console.log(new_var);
</script>
```

Разберём более подробно, что происходит в коде примера **example_10**:

- объявление **function declaration** создаёт функцию и помещает её в переменную с именем **discr**;
- далее мы скопировали значение функции в переменную **new_var**. Обратите внимание на отсутствие круглых скобок после **discr**. Если бы



они были, то выражение:

```
var new_var = discr;
```

записало бы результат вызова **discr**() в переменную **new_var**, а не сам исходный код функции **discr**;

- теперь функция может вызываться как **discr**(), так и **new_var**().

> ***Примечание***. Работа с исходным кодом функции как значением переменной – важная особенность языка JS и будет часто использоваться в дальнейшем. Подробнее в следующих уроках темы.

Размещение функций

Чтобы основной файл был избавлен от излишнего нагромождения программных конструкций, объявления функций принято выносить в отдельные файлы (впоследствии объединять в модули).

Вынося определения функций во внешние файлы, не забудьте эти файлы подключить:

```
<!-- подключаем файл с функцией -->

<script src="function.js"></script>
```

И не забывайте, если функция взаимодействует с деревом DOM, она не может быть вызвана раньше, чем это дерево будет построено.

example_11. index.html – выносим определение функции во внешний файл

```
<!-- ... -->
    <title>Программирование в среде JavaScript</title>
    <!-- подключаем файл с функцией -->
    <script src="function.js"></script>
</head>
<body>
```

```
    <header>
        <h1 align="center">Функции JavaScript</h1>
    </header>
    <script>
        // вызов функции welcome()
        welcome();
    </script>
<!-- ... -->
```

В демонстрационном примере **example_11** представлен код функции **welcome**(), подключаемой из внешнего файла **function.js**.

example_11. function.js – файл определения функции

```
<script>
    // объявление многострочной стрелочной функции
    var welcome = () => {
        let user = prompt("Представьтесь, пожалуйста");
        if (user == null ||  user.replace(/\s/g, '') == "")
            document.write ("<h1>Здрасссьте вам ...</h1>");
        else
            document.write ("<h1>Добро пожаловать, уважаемый ",
            user, "!</h1>");
    }
</script>
```



Особенности функциональных выражений

Ссылка на функцию внутри тела функции

Функциональное выражение может быть **именованным** и **анонимным**. С анонимными выражениями разобрались, посмотрим, когда могут использоваться именованные выражения.

Если необходимо сослаться на текущую функцию **внутри тела этой функции**, то нужно создать **именованное функциональное выражение**. Имя будет локальным (только для тела функции, её области видимости).

Самый яркий пример использования ссылки на текущую функцию внутри тела этой функции - **рекурсивный вызов**.

example_12. Рекурсивный вызов именованного функционального выражения

```
<script>
    // общий вид вычисления факториала
    // n! = 1 * 2 * 3 * 4 * ... * n
    // факториал числа 5
    // 5! = 1 * 2 * 3 * 4 * 5 (=120)
    // именованная функция expression
    var fact = function fnFact (num) {
        if (num >= 1) {
            console.log(num);
            // рекурсивный вызов именованной функции,
            // где fnFact - имя функции
            return num * fnFact(num - 1);
```

```
        };
        // выход из рекурсии
        return 1;
    };
    res = fact(5);
    console.log("==========");
    console.log("5! = ", res);
</script>
```

Свойство name функционального выражения

Переменная, которой присвоено функциональное выражение, будет иметь свойство **name**, содержащее имя функции, и которое **не изменяется** при переприсваивании другой переменной:

- для **анонимной** функции значением свойства **name** будет **имя переменной** (неявное имя);
- для **именованной** функции будет использовано **имя функции** (явное имя).

Следующий пример основан на том, что функция по сути является объектом. Звучит немного запутано, но на то она и практика, чтобы разобраться.

example_13. Свойство name функционального выражения

```
<script>
    // анонимное выражение
    var cube = function (num) {
        return num * num * num;
    };
    // вывод свойства name
```

```
    console.log(cube.name); // cube
    // переприсвоим значение новой переменной
    var varCube = cube;
    // вывод свойства name новой переменной
    console.log(varCube.name); // cube
    // именованное выражение
    var cube = function fnCube (num) {
        return num * num * num;
    };
    // вывод свойства name
    console.log(cube.name); // fnCube
    // переприсвоим значение новой переменной
    var varCube = cube;
    // вывод свойства name новой переменной
    console.log(varCube.name); // fnCube
</script>
```

Подъем функции в области видимости

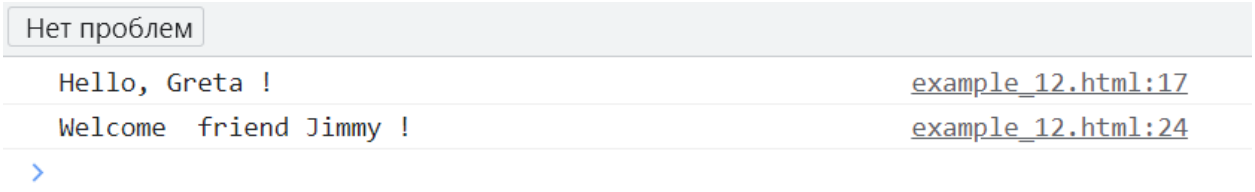
Функции, объявленные как **function declaration** (инструкция), всегда поднимаются наверх в текущей области видимости.

example_14. Объявление функции в function declaration

```
<script>
    // два представленных примера эквивалентны
    // 1. - вызов, объявление
    // 2. - объявление, вызов
```

```
    // вызов функции sayHello
    sayHello("Greta");
    // объявление функции sayHello
    function sayHello (name) {
        console.log('Hello,', name, '!');
    }
    // ====================
    // объявление функции myGreeting
    function myGreeting (name) {
        console.log('Welcome  friend', name, '!');
    };
    // вызов функции myGreeting
    myGreeting ("Jimmy");
</script>
```

Несмотря на разный порядок объявления и вызова, обе функции отработали без ошибок.



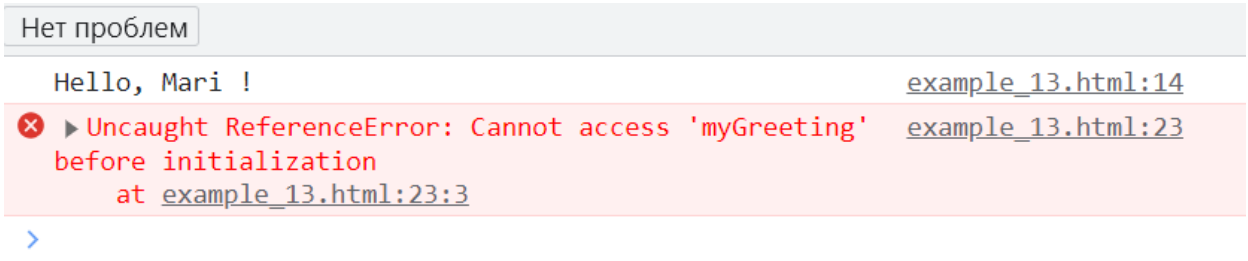
Если же функция объявляется как **function expression** (выражение), то такая функция не поднимается в текущей области видимости.

example_15. Объявление функции в function expression

```
<script>
    // два представленных примера НЕ эквивалентны
    // 1. - объявление, вызов
```

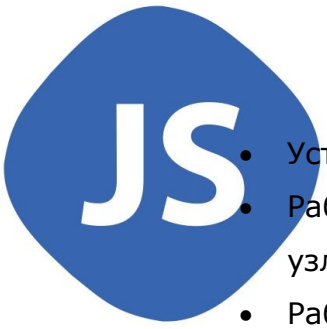
```
    // 2. - вызов, объявление
    // объявление функции sayHello
    const sayHello = function (name) {
        console.log('Hello,', name, '!');
    }
    // вызов функции sayHello
    sayHello("Mari");
    // ====================
    // вызов функции myGreeting
    myGreeting("John");
    // объявление функции myGreeting
    const myGreeting = function (name) {
        console.log('Welcome  friend', name, '!');
    }
</script>
```

В функциональных выражениях порядок объявления функции имеет значение.



Из-за этих двух возможных вариантов подъем функции потенциально может сбить с толку, поэтому не рекомендуется к применению на практике. Всегда сначала **объявляйте функции** перед тем, как их **использовать**.

В **самостоятельных работах** выполняем задания:



- Устранение ошибки вывода результатов работы функции.
- Работа функции-процедуры с элементами массива и коллекциями HTML-узлов.
- Работа функции-процедуры с деревом DOM.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

